

Artifact: OE25_{dev} Framework

0.1 Purpose

OE25Dev is a dataset of *developer-written Java unit tests* designed to support research on test oracle generation and evaluation. Unlike prior benchmarks that rely mainly on automatically generated tests, OE25Dev focuses on realistic test suites written and maintained by developers in real-world open-source projects.

The dataset enables reproducible evaluation of oracle generation techniques, including large language model (LLM)-based approaches, under practical testing conditions.

0.2 What Is New

OE25Dev introduces a complete, end-to-end framework for constructing and evaluating test oracles on developer-written test suites. In addition to providing decomposed single-oracle test cases, OE25Dev supports mutation-based evaluation and systematic comparison between original, decomposed, and tool-generated oracles.

Figure 1 illustrates the overall OE25Dev pipeline. The process starts by downloading the target projects and collecting developer-written test cases. Each test case is then decomposed by splitting multi-oracle tests into single-oracle instances while preserving the full test prefix. Non-compilable test cases are removed, and the remaining tests are executed to obtain baseline results.

The resulting dataset forms the final OE25Dev benchmark with decomposed test cases. Mutation testing is then applied to (i) the original projects, (ii) the OE25Dev decomposed tests, and (iii) test suites augmented with tool-generated oracles. Mutation testing results are collected and saved separately for each setting, enabling direct comparison of oracle effectiveness.

This pipeline enables realistic evaluation of oracle generation tools under developer-written test case scenarios, supporting analyses such as mutation kill rates, false positives, and unique bug identification.

0.3 Artifact Organization

The artifact is organized as follows:

- **projects_original/**: Contains all original open-source project versions used in this artifact. These projects are unmodified and include the original developer-written test suites.

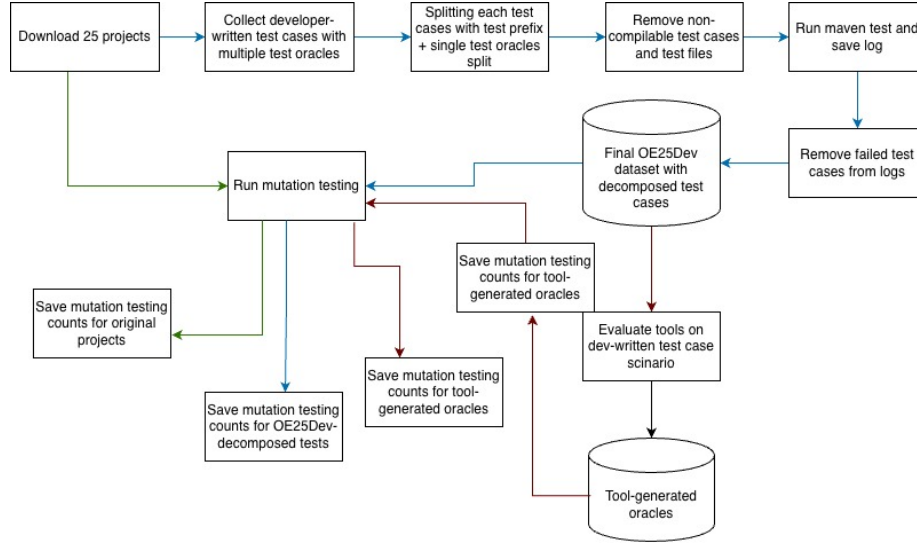


Figure 1: Overview of the OE25Dev pipeline. The pipeline shows project preparation, test decomposition, mutation testing, oracle generation, and evaluation across original, decomposed, and tool-generated test suites.

- **projects_decompose/**: Used for constructing the OE25Dev dataset. This directory contains projects with decomposed test cases, where each test contains a single oracle. Generated test files use the suffix `_OE25Dev.java`.
- **scripts/**: Contains all scripts required to run the OE25Dev framework, including dataset construction, test execution, mutation testing, oracle removal, oracle injection, and tool evaluation.

All other directories and files are automatically generated when the provided scripts are executed.

0.4 Usage Guidelines

This section describes the recommended steps to use the OE25Dev dataset and framework.

Step 1: Download Projects If the projects are not already available, download them using:

```
scripts/download_projects.sh
```

The downloaded projects are stored in `projects_original/`.

Step 2: Run Original Test Suites Execute the original test suites and collect baseline statistics using:

```
scripts/Final_mvn_run_original.sh
```

This step builds each project and records the number of executed tests.

Step 3: Build OE25Dev Decomposed Tests Construct the OE25Dev dataset using:

```
scripts/main.sh
```

This script reads `inputs.csv` and `meta.csv`, decomposes multi-oracle test methods into single-oracle test cases, and generates new test files with the suffix `_OE25Dev.java`.

Step 4: Run Decomposed Test Suites Execute the decomposed test suites using:

```
scripts/final_mvn_run_decomposed.sh
```

This step records test execution statistics for the OE25Dev test cases.

Step 5: Mutation Testing Run mutation testing using PIT:

```
scripts/run_pitest.sh
```

Mutation testing logs are saved, and final mutation statistics are recorded in `mutation_count.csv`.

Step 6: Oracle Removal Prepare oracle-free test cases using:

```
scripts/remove_assertion.py
```

All assertion-based and exception-based oracles are removed and replaced with a placeholder token.

Step 7: Oracle Injection After generating oracles, inject them into the test cases using:

```
scripts/inject_generated_assertions.py
```

Step 8: Tool Evaluation Evaluate generated oracles using:

```
scripts/evaluate_tool.sh
```

This step compiles and executes the modified test suites, comments out tests with compilation errors, and records test execution results, false positives, mutation kills, and unique bug identification statistics.

OE25Dev provides a realistic and extensible benchmark for evaluating test oracle generation techniques on developer-written test suites. By decomposing tests at the oracle level and supporting a complete evaluation pipeline, OE25Dev enables systematic and fine-grained analysis of oracle quality in real-world software projects.